

高性能科学计算课程汇报

组员：杨译坤、刘汇川、刘尚坤
时间：2026年6月10日



第11组汇报 (17:34~17:46)

CloverLeaf 并行计算 程序分析与测试

高性能计算课程报告—第一部分

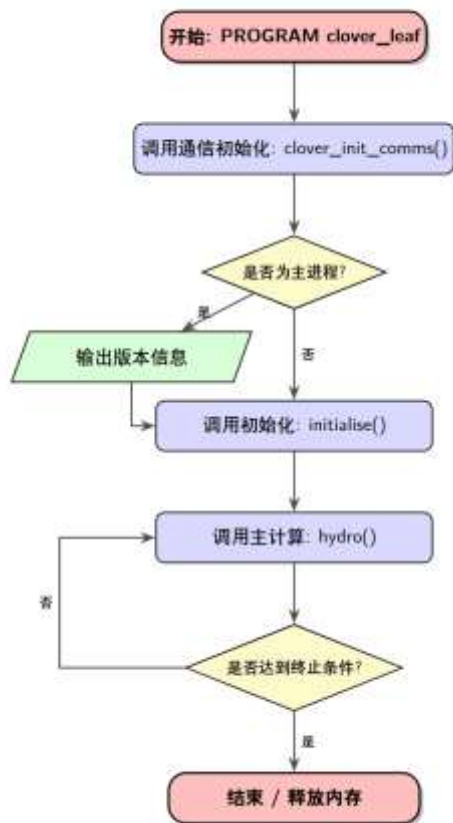
Contributors

杨译坤：1、2小题内容贡献，PPT制作与汇报
刘汇川：3、4、5、6、7小题内容贡献



1. 程序解读（主程序over_leaf）

第3页



核心物理模型与数值方法

控制方程：二维可压缩欧拉方程，描述流体的质量、动量和能量守恒。

数值方法：

- ✓ 采用交错笛卡尔网格：密度、压力、内能定义在单元中心，速度定义在顶点。
- ✓ 时间推进采用二阶预测 - 校正拉格朗日步
- ✓ 典型应用场景：模拟激波、接触间断等流体力学现象，是典型的访存密集型科学计算程序。

1. 程序解读（主计算hydro()）

第4页



拉格朗日阶段

网格随流体运动，依次执行预测物理量、计算速度加速度、校正物理量等步骤，完成物理量的时步推进，是流体动力学核心求解过程。

平流重映阶段

通过通量计算和对流项处理，将变形网格上的质量、动量和能量场重新映射回规则的欧拉网格，消除网格畸变；最后计算下一时刻的时间步长，进入下一轮迭代。

2. 程序编译与性能分析 (clover_bm_short.in)

第5页

指标	Serial	MPI	OpenMP	Hybrid
配置	1 进程	4 进程	4 线程	2 进程 × 2 线程
用户时间 (s)	24.90	17.13	18.16	17.60
系统时间 (s)	0.68	0.64	0.20	0.36
实际运行时间 (s)	25.53	5.18	4.70	4.89
CPU 使用率 (%)	100%	343%	390%	367%
主动上下文切换	86	5,531	95	2,464
被动上下文切换	27	149	18	215
峰值内存 (KB)	726,704	62,252	184,976	108,516
加速比	1.00	4.93	5.43	5.22
并行效率 (%)	100%	123%	136%	131%

3. 算例程序正确性验证

第6页

算例	网格	步数	偏差	结果
test 2	960×960	87	$1.14 \times 10^{-13}\%$	✅ PASSED
test 3	960×960	2955	$1.79 \times 10^{-2}\%$	NOT PASSED
test 4	3840×3840	87	$1.58 \times 10^{-11}\%$	✅ PASSED
test 5	3840×3840	2955	$4.88 \times 10^{-10}\%$	✅ PASSED

1. 运行 **CloverLeaf** 提供的标准输入文件，进行程序正确性验证
2. 程序内置自检机制（`field_summary.f90` 第 154-158 行）：
 - 程序自动计算算出的 **KE** 与精确参考值的百分比偏差
 - 若偏差 $< 0.001\%$ ，自动判定为 **PASSED**

4. 不同网格规模并行效率测试 (clover_bm_short.in)

版本	配置	总单元	时间(秒)	加速比	效率
MPI	1p	1	8.44	1.00	100%
	2p	2	4.02	2.10	105%
	4p	4	2.86	2.95	74%
	8p	8	1.61	5.24	66%
OpenMP	1t	1	9.87	1.00	100%
	2t	2	5.10	1.94	97%
	4t	4	3.59	2.75	69%
	8t	8	2.79	3.54	44%
Hybrid	1p2t	2	4.80	2.13	107%
	1p4t	4	4.52	2.27	57%
	2p1t	2	5.22	1.96	98%
	2p2t	4	3.92	2.61	65%
	2p4t	8	3.13	3.27	41%
	4p1t	4	3.34	3.07	77%
	4p2t	8	2.70	3.79	47%

现象分析：

1. MPI 8进程效率 66% > OpenMP 8线程效率 44%。

原因：OpenMP受共享内存带宽竞争、伪共享、线程同步开销影响更大

2. 超线性加速比 (MPI 2p效率105% , Hybrid 1p2t效率107%)

- 数据拆分后单节点缓存命中率提升

- 串行版本因内存/缓存压力导致基线时间偏大

3. 混合编程表现

- 最佳配置：4p2t (加速比3.79 , 效率47%)

- 整体未显著优于纯MPI

- 原因：CloverLeaf网格拆分后，单进程内OpenMP并行粒度有限，通信仍由MPI主导

4. 不同网格规模并行效率测试

OpenMP

算例	OMP=1	OMP=2	OMP=4	OMP=8	最优
bm2	412.74	308.37	268.14	260.38	268.14 (OMP=4)
bm4	828.60	625.85	551.62	550.22	550.22 (OMP=8)
bm8	1621.23	1274.57	1131.65	1126.20	1126.20 (OMP=8)

MPI

算例	np=1	np=2	np=4	np=8	最优
bm2	399.07	298.50	258.87	255.33	258.87 (np=4)
bm4	805.41	611.13	539.58	539.15	539.15 (np=8)
bm8	1576.46	1218.76	1090.00	1095.48	1090.00 (np=4)

Serial 串行基准

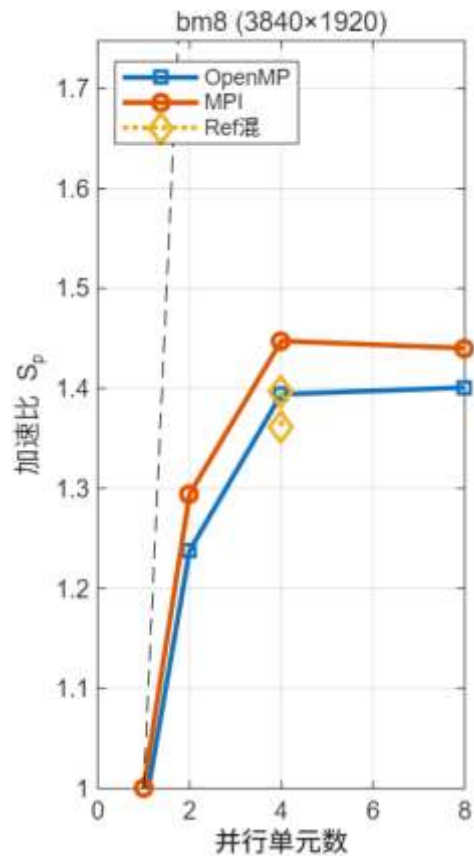
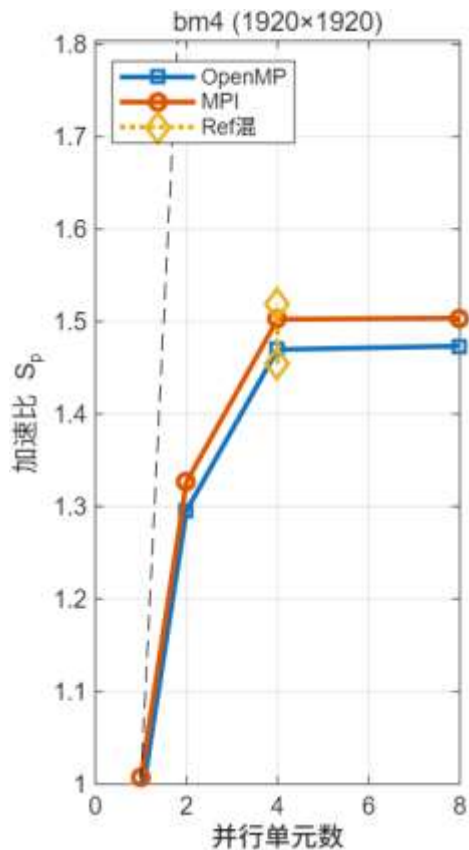
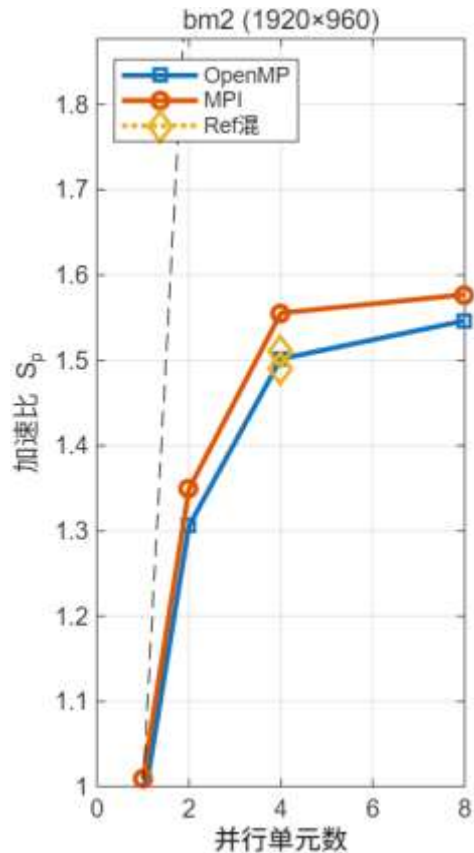
算例	网格	单元数	Wall Clock (s)
bm2	1920×960	1.84M	402.66
bm4	1920×1920	3.69M	810.73
bm8	3840×1920	7.37M	1577.74

Ref 混合 (MPI+OpenMP)

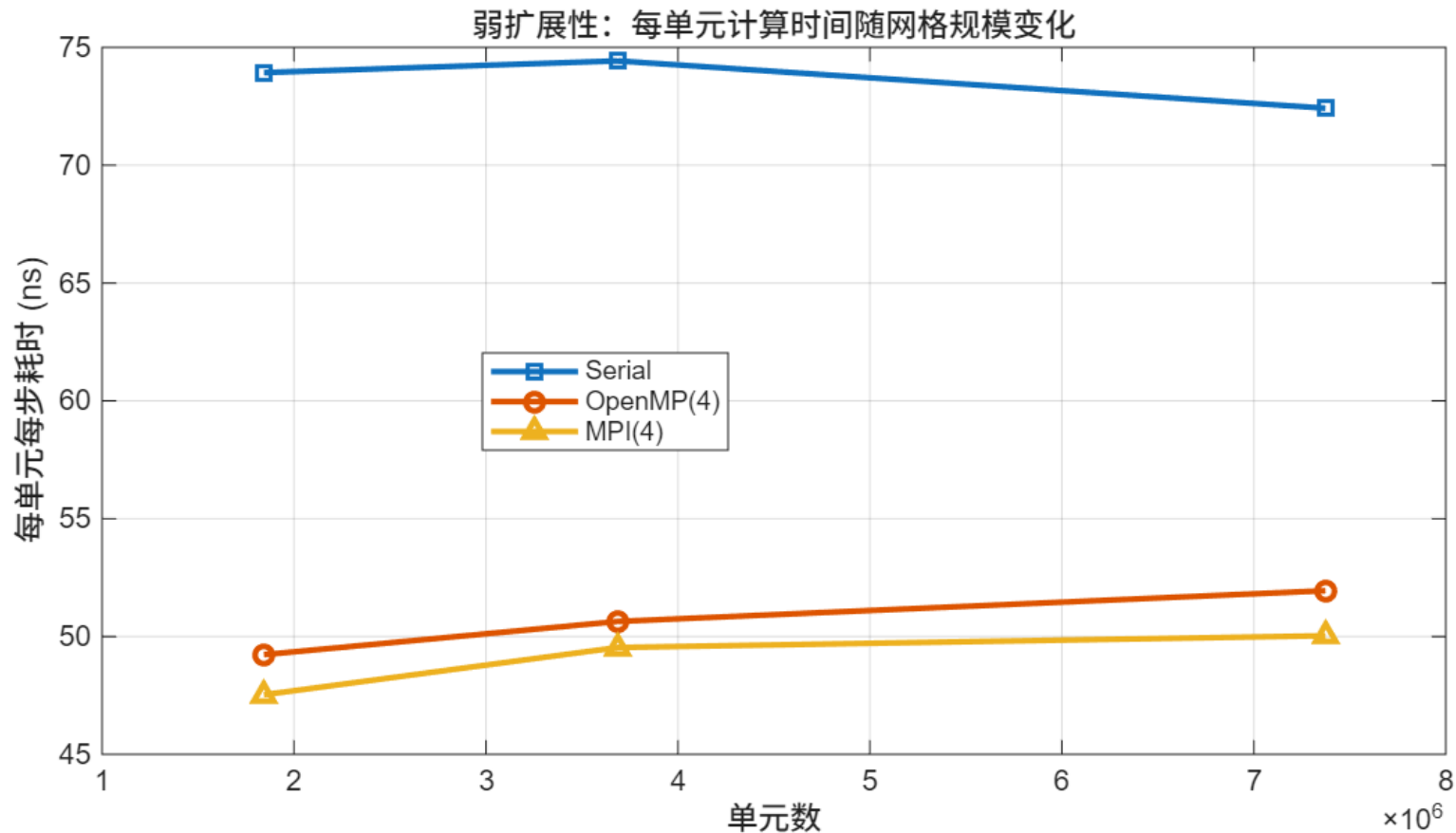
算例	np=1 OMP=2	np=2 OMP=1	np=1 OMP=4	np=2 OMP=2	np=4 OMP=1	np=2 OMP=4	np=4 OMP=2
bm2	408.20	304.35	411.75	308.76	266.66	309.73	270.10
bm4	823.04	626.23	824.59	623.64	557.38	631.15	533.70
bm8	1503.52	1260.44	1769.34	1253.32	1129.03	1261.25	1159.05

4. 不同网格规模并行效率测试

第9页

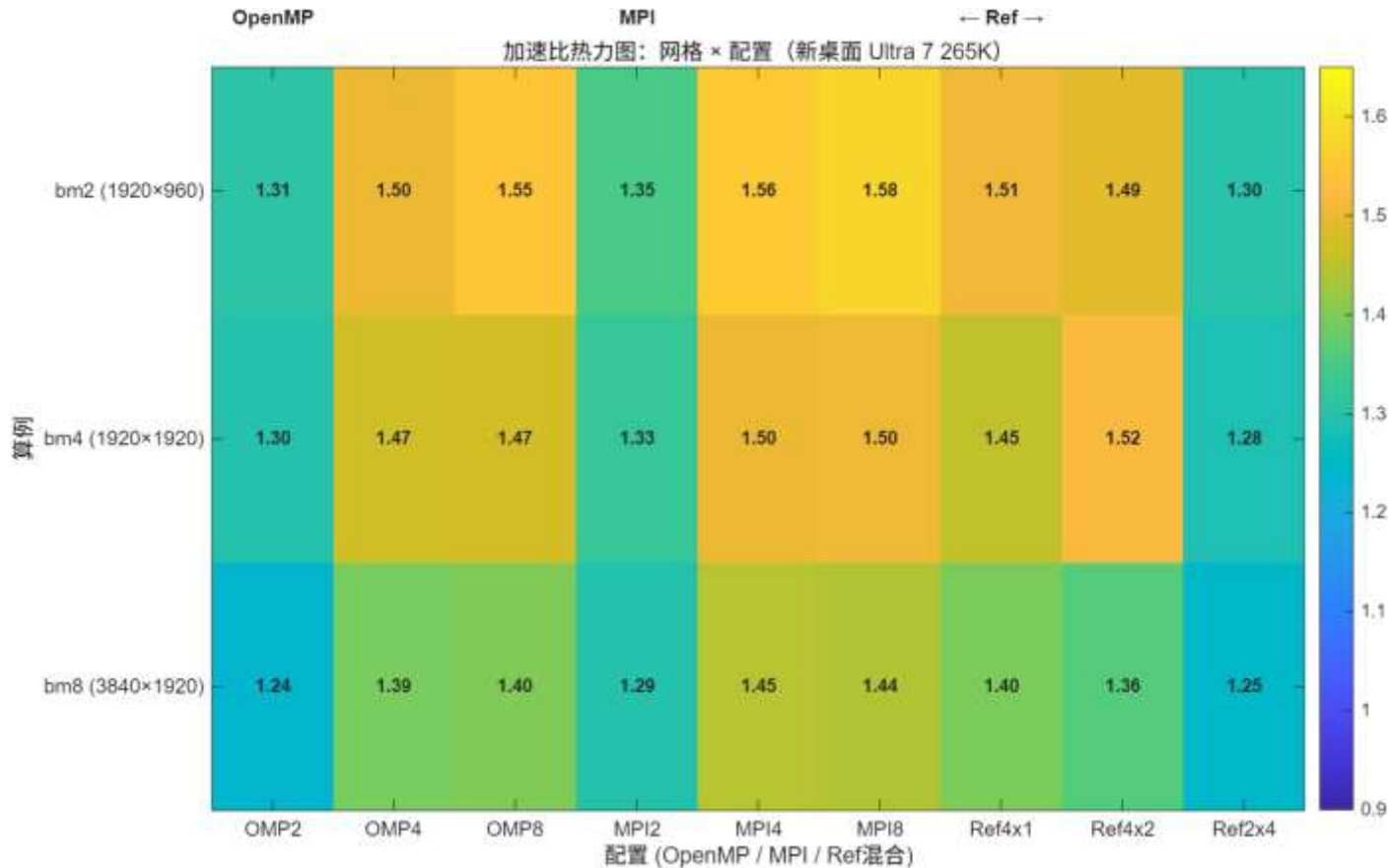


4. 不同网格规模并行效率测试



4. 不同网格规模并行效率测试

第11页



5. CUDA版本编译与适配

第12页

序号	修改位置	原内容	改为	原因类别
①	第 91 行	无 Ampere 定义	新增 <code>CODE_GEN_AMPERE=-gencode arch=compute_86,code=sm_86</code>	GPU 架构适配
②	第 84 行	<code>NV_ARCH=KEPLER</code>	<code>NV_ARCH=AMPERE</code>	默认架构切换
③	第 102 行	<code>\$(NV_OPTIONS)</code>	<code>\$(NV_OPTIONS) --allow-unsupported-compiler</code>	CUDA 12 与 GCC 版本兼容
④	第 42 行	<code>FLAGS_GNU = -O2</code>	<code>FLAGS_GNU = -O2 -fallow-argument-mismatch</code>	gfortran 10+ 接口严格检查兼容
⑤	编译环境	无	<code>export CUDA_HOME=/usr/lib/cuda</code>	WSL apt 安装的非标准路径适配

原 Makefile 仅支持 NVIDIA Fermi (计算能力 2.0/2.1) 和 Kepler (计算能力 3.0/3.5) 两代老旧架构，且未考虑现代 GNU 编译器与 CUDA 12.x 之间的兼容性适配。因此进行 4 处修改并设置 1 项环境变量。

6. CUDA正确性与性能对比

第13页

算例	网格	单元数	CPU 串行 (s)	CPU 8核最优 (s)	CUDA GPU (s)	GPU/串行	GPU/8核
clover_bm	960×960	0.92M	309.9	—	68.08	4.55×	—
bm2	1920×960	1.84M	654.7	528.1	136.13	4.81×	3.88×
bm4	1920×1920	3.69M	1247.8	1146.9	272.20	4.58×	4.21×
bm8	3840×1920	7.37M	2498.6	2108.2	544.90	4.59×	3.87×

指标	结论
正确性	test 2/4 PASSED, test 3 存在与 CPU 版一致的编译器/架构浮点差异 (物理结果正确)
GPU 绝对加速比	4.5–5.3× (相对 CPU 单核)
GPU 相对 8 核 CPU	3.8–4.2× (RTX 3050 Laptop 单卡 ≈ 30 CPU 核等效)
每单元每步耗时	GPU: 2.50×10^{-8} s, CPU 串行: 1.15×10^{-7} s, GPU 每单元快 4.6 倍
瓶颈本质	GPU 同样受 Memory Bound 约束, 但 192 GB/s 独立显存远优于 CPU 双通道 DDR4, 这是加速的根本来源
扩展性	GPU 端 $O(N)$ 线性度极佳 ($R^2 \approx 1$), 4 GB 显存足以容纳 bm8 及以下规模的所有数据

- test2 / test4 完全通过
- 不是GPU核心多，而是显存带宽赢了内存带宽（GPU通过独立显存通道将内存带宽瓶颈从“单通道”提升到了“聚合高带宽”层次）

7. CUDA编程技巧总结

第14页

技巧	核心 CUDA 特性	解决的工程问题
细粒度线程映射	<code><<<grid, block>>></code> 启动配置	暴露充分并行度，最大化 GPU 占用率
CUDALAUNCH 宏	CUDA Events, <code>cudaGetLastError</code>	统一 kernel 启动、计时与错误检查
全局内存预分配	<code>cudaMalloc</code> + <code>cudaMemset</code>	消除运行时分配开销，避免内存泄漏
树形归约	共享内存 + <code>__syncthreads</code>	$O(\log N)$ 高效全局统计
cell_info 类型标记	C++ <code>struct</code> 参数化	一份代码处理四种网格数据类型
MPI+CUDA Halo 交换	<code>cudaMemcpy</code> + <code>MPI_ISEND</code>	GPU↔CPU↔网络的三级数据搬运流水线
多 GPU 轮询绑定	<code>cudaSetDevice</code>	多卡环境下的负载均衡
Fortran-C 索引互译	<code>FTNREF2D</code> 宏	零拷贝的跨语言数组访问
工作数组消除依赖	6 个 <code>work_array</code>	以空间换时间，解除 kernel 内部数据依赖

- CloverLeaf 的 CUDA 版本将全部 18 个流体力学计算 kernel 移植到 GPU 上执行。CUDA 代码约 3500 行 (`.cu` + `.cuknl` + `.hpp`) , 通过 Fortran-C 混合编译与原有 Fortran 框架集成。以下按主题总结代码中运用的关键 CUDA 编程技巧。



GitHub

软件源代码托管服务平台

二维 Laplace 方程的 MPI+CUDA 并行求解

多种迭代方法 · 两个方向 MPI 并行

CUDA GPU 加速 · 异步 Stream IO

高性能计算课程报告—第二部分

Contributor: 刘尚坤

演示时间大约需要 5 分钟

运行环境与版本信息

https://github.com/Liskelleo/hpsc_assignment/laplace_mpi_cuda

1

操作系统

WSL2 Ubuntu-HPC / Ubuntu 20.04.6 LTS

2

CPU

Intel Xeon Silver 4210R @ 2.40GHz , 10核/20线程

3

MPI

OpenMPI 4.0.3 (MPI-3.1) + mpicxx + OpenMP -O3

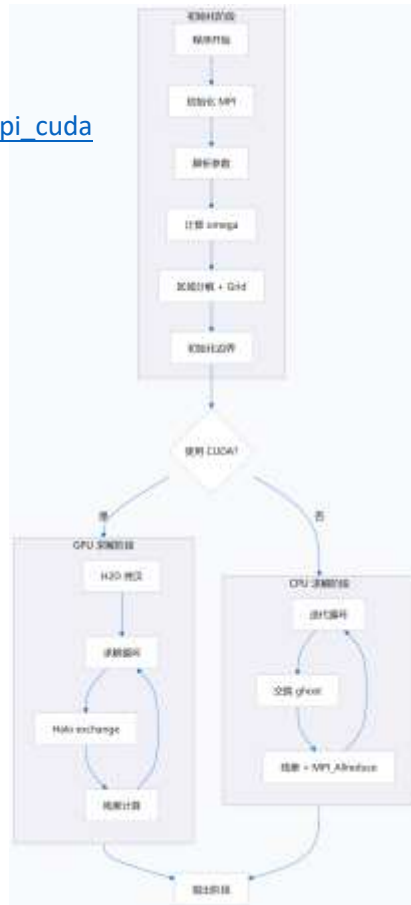
4

CUDA / GPU

CUDA 12.x / NVIDIA RTX A2000 (Ampere, sm_86, 3584 核)

源代码结构

```
laplace_mpi_cuda/
├── src/
│   ├── laplace_solver.h
│   ├── laplace_solver.cpp
│   ├── laplace_cuda.cu
│   ├── mpi_io.cpp
│   ├── stream_async.cu
│   └── main_single_cuda.cu
├── main.cpp
├── Makefile
├── run_tests.sh
├── scaling_compare.sh
├── size_scaling.sh
├── bench_pinned.sh
├── check_mpi_io.py
├── bin/
└── visualize.py
```



数学模型与边界条件

Laplace 方程

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0, \quad (x, y) \in [0, 1] \times [0, 1]$$

Dirichlet 边界条件

$$\begin{aligned} T(0, y) &= 0, & T(1, y) &= 0, \\ T(x, 0) &= 0, & T(x, 1) &= 1. \end{aligned}$$

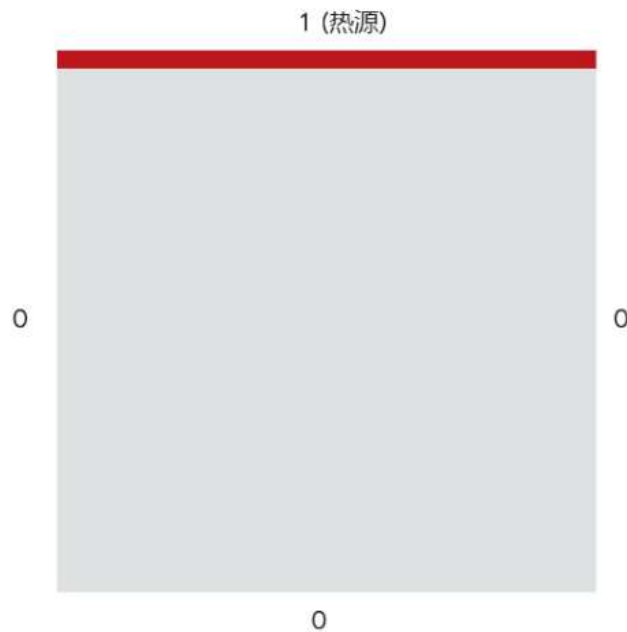
有限差分离散 (规定最大迭代步数)

均匀网格: $dx = dy = 1/(N+1)$

三种迭代格式: Jacobi / GS / SOR

① 收敛准则: $|u^{k+1} - u^k|_{\infty} < 10^{-6}$

算例的初始条件



有限差分离散

采用中心差格式对Laplace方程进行离散，将求解域划分为 $(N_x + 1) \times (N_y + 1)$ 个网格点，其中 N_x 和 N_y 分别为 x 和 y 方向的内点数量。网格步长为：

$$\Delta x = \frac{1}{N_x + 1}, \quad \Delta y = \frac{1}{N_y + 1}$$

记 $u_{i,j}$ 为网格点 $(i\Delta x, j\Delta y)$ 处的温度值，利用Taylor展开：

$$u_{i+1,j} = u_{i,j} + \Delta x \left. \frac{\partial u}{\partial x} \right|_{i,j} + \frac{(\Delta x)^2}{2} \left. \frac{\partial^2 u}{\partial x^2} \right|_{i,j} + \frac{(\Delta x)^3}{6} \left. \frac{\partial^3 u}{\partial x^3} \right|_{i,j} + O((\Delta x)^4)$$

$$u_{i-1,j} = u_{i,j} - \Delta x \left. \frac{\partial u}{\partial x} \right|_{i,j} + \frac{(\Delta x)^2}{2} \left. \frac{\partial^2 u}{\partial x^2} \right|_{i,j} - \frac{(\Delta x)^3}{6} \left. \frac{\partial^3 u}{\partial x^3} \right|_{i,j} + O((\Delta x)^4)$$

两式相加，消去一阶导数项，得到二阶导数的中心差分近似：

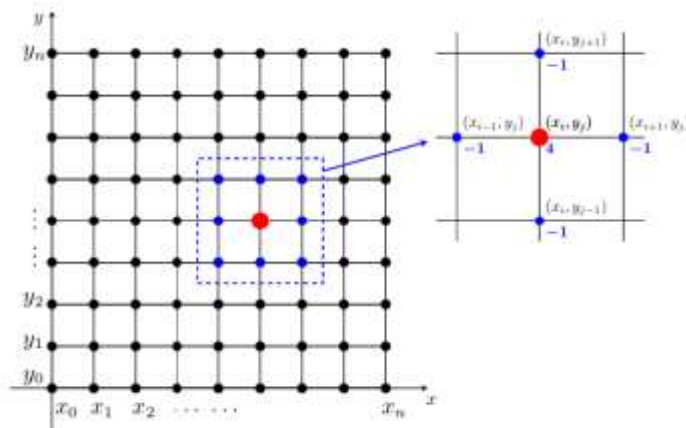
$$\left. \frac{\partial^2 u}{\partial x^2} \right|_{i,j} = \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{(\Delta x)^2} + O((\Delta x)^2)$$

同理可得 y 方向二阶导数近似。代入原方程，忽略高阶项，得到离散后的方程：

$$\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{(\Delta x)^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{(\Delta y)^2} = 0$$

当 $\Delta x = \Delta y = h$ (均匀网格) 时，整理得到**五点差分格式**：

$$u_{i,j} = \frac{1}{4}(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1})$$



迭代算法与红黑排序

Jacobi迭代

$$u_{i,j}^{(k+1)} = \frac{1}{4}(u_{i+1,j}^{(k)} + u_{i-1,j}^{(k)} + u_{i,j+1}^{(k)} + u_{i,j-1}^{(k)})$$

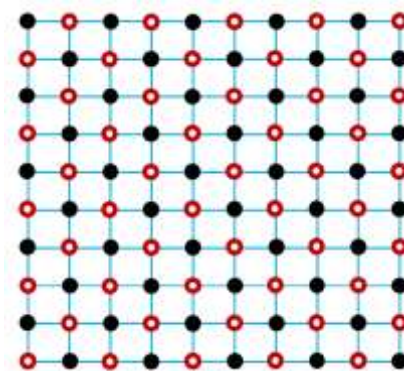
Gauss-Seidel迭代

$$u_{i,j}^{(k+1)} = \frac{1}{4}(u_{i+1,j}^{(k)} + u_{i-1,j}^{(k+1)} + u_{i,j+1}^{(k)} + u_{i,j-1}^{(k+1)})$$

Successive Over-Relaxation迭代 (SOR)

$$u_{i,j}^{(k+1)} = (1 - \omega)u_{i,j}^{(k)} + \frac{\omega}{4}(u_{i+1,j}^{(k)} + u_{i-1,j}^{(k+1)} + u_{i,j+1}^{(k)} + u_{i,j-1}^{(k+1)})$$

观察五点差分格式可以发现：红点的邻居（上下左右）均为黑点，黑点的邻居均为红点。因此，采用红黑排序策略，将二维网格点按 $i + j$ 的奇偶性分为两类：**红点**： $i + j$ 为偶数，**黑点**： $i + j$ 为奇数。并行迭代方案分为两步：



1. **第一步**：并行更新所有红点，其邻居均为黑点旧值，互不依赖

2. **第二步**：并行更新所有黑点，其邻居均为红点新值，互不依赖

在计算过程中，对 $u_{i,j}^{(k+1)}$ 的值进行更新时，我们可以先更新红色节点上的值，此时所使用的只是黑色节点的数据，然后再更新黑色节点上的值，这时使用的是红色节点的数据。由于在更新红点时，各个红点之间是相互独立的，因此可以并行计算。同样，在更新黑点时，各个黑点之间也是相互独立的，因此也可以并行计算。

算法	谱半径	迭代次数	并行性
Jacobi	$1 - O(N^{-2})$	$O(N^2)$	并行
Gauss-Seidel	$1 - O(N^{-2})$	$O(N^2)$	难以并行
SOR	$1 - O(N^{-1})$	$O(N)$	难以并行
SOR红黑	$1 - O(N^{-1})$	$O(N)$	并行

第一轮优化

算法选择：Jacobi → G-S → SOR

$$u_{i,j}^{(k+1)} = (1-\omega)u_{i,j}^{(k)} + \frac{\omega}{4}(u_{i+1,j}^{(k)} + u_{i-1,j}^{(k)} + u_{i,j+1}^{(k)} + u_{i,j-1}^{(k)})$$

最优松弛因子理论分析：对于五点差分格式离散化的Poisson方程，SOR迭代矩阵的谱半径在最优松弛因子 ω_{opt} 处达到最小。理论推导表明，Jacobi迭代矩阵的谱半径为 $\rho(B_J) = \cos(\pi/(N+1))$ ，而最优松弛因子满足：

$$\frac{\omega_{opt}\rho(B_J)}{2-\omega_{opt}} = \omega_{opt} - 1$$

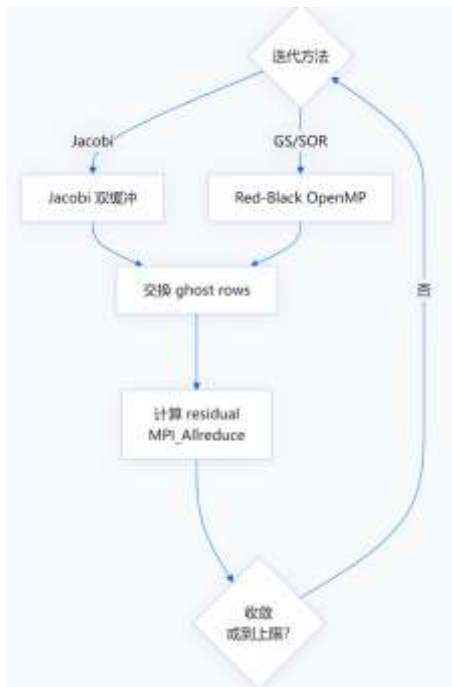
解得：

$$\omega_{opt} = \frac{2}{1 + \sqrt{1 - \rho(B_J)^2}} = \frac{2}{1 + \sin(\frac{\pi}{N+1})}$$

此时SOR迭代矩阵的谱半径为：

$$\rho(B_{SOR}) = \omega_{opt} - 1 = \frac{1 - \sin(\pi/(N+1))}{1 + \sin(\pi/(N+1))} \approx 1 - \frac{2\pi}{N+1}$$

由此可知，SOR算法的收敛速度从Jacobi的 $O(N^2)$ 提升至 $O(N)$ ，即迭代次数与网格边长成正比而非平方关系。本程序在运行时根据网格规模动态计算 ω_{opt} ，无需用户指定。



Jacobi: (512×512)

指标	值
耗时	13.46 s
迭代步数	50000
最终残差	3.83e-6

⚠ 谱半径 $\rho \approx 0.99992$ ，每步残差仅减少 0.008%，收敛较慢。

优化后: (128×128)

方法	耗时	步数	残差
Jacobi	0.0189 s	2000	1.11e-4
G-S	0.0676 s	2000	8.48e-5
SOR红黑	0.0215 s	300	1.32e-7

☑ 传统的SOR算法由于数据依赖关系，难以并行化，采用红黑排序算法，300步即达 $1e-7$ 精度。

第二轮优化

CUDA GPU 加速

设计要点：

Red-Black
SORCoalesced
内存仅交换
Ghost行

线程块 $\text{dim3}(16,16) = 256$ 线程/块；

Red-Black 分色保证同色点完全并行；

共享内存树形归约计算 L_∞ 残差，避免 `atomicMax` 序列化。

调试过程中发现并修复的问题：

问题	原因	修复
架构不匹配	<code>-arch=sm_75</code> 在 Ampere 上生成低效代码	改为 <code>-arch=sm_86</code>
共享内存竞争	<code>tid=threadIdx.x(0~15)</code> , 16个线程写同一slot	改为 <code>tid = threadIdx.y*blockDim.x + threadIdx.x(0~255)</code>
H2D缺少同步	<code>cudaMemcpyAsync</code> 后无 <code>sync</code> , kernel读旧数据	改用同步 <code>cudaMemcpy</code>
全量传输	每迭代D2H+H2D全数组 (2.1MB), PCIe成瓶颈	改为仅传Ghost行/列 (4KB), 减少500×传输

1024×1024 单进程对比

版本	耗时	相对CPU
CPU SOR	12.7470 s	1×
CUDA SOR	0.5551 s	22.96×

📌 在较大规模、单进程GPU场景下，CUDA加速明显。

第三轮优化

Pageable host buffer 会导致 Host/Device 传输需要 staging copy。
早期列 ghost 还包含临时 cudaMalloc/cudaFree，在多方向通信时开销明显。

Pinned Memory Ghost Exchange

1. 预分配 pinned host buffer，避免 pageable 内存分页开销
2. 预分配 device column buffer，消除每步临时分配
3. 减少 H2D / D2H 传输中的 staging 拷贝

程序优化

在 grid 中预分配 pinned host buffer 与 device column buffer:

```
double *h_pin_row_snd;
double *h_pin_row_rcv;
double *h_pin_col_snd;
double *h_pin_col_rcv;
double *d_col_l;
double *d_col_r;
```

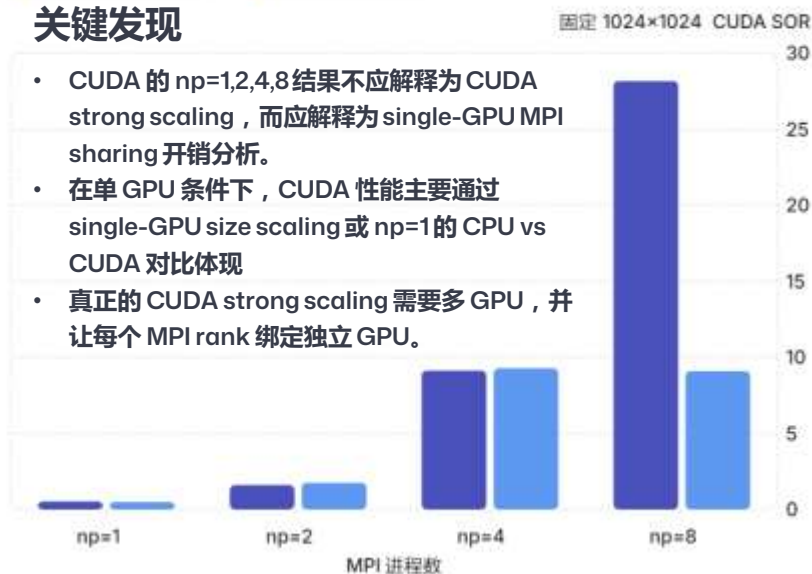
首次 CUDA ghost exchange 时分配:

```
cudaMallocHost(&grid->h_pin_row_snd, buf_size * sizeof(double));
cudaMallocHost(&grid->h_pin_row_rcv, buf_size * sizeof(double));
cudaMallocHost(&grid->h_pin_col_snd, buf_size * sizeof(double));
cudaMallocHost(&grid->h_pin_col_rcv, buf_size * sizeof(double));
cudaMalloc(&grid->d_col_l, ny_local * sizeof(double));
cudaMalloc(&grid->d_col_r, ny_local * sizeof(double));
```

■ Pageable Memory ■ Pinned Memory

关键发现

- CUDA 的 np=1,2,4,8 结果不应解释为 CUDA strong scaling，而应解释为 single-GPU MPI sharing 开销分析。
- 在单 GPU 条件下，CUDA 性能主要通过 single-GPU size scaling 或 np=1 的 CPU vs CUDA 对比体现
- 真正的 CUDA strong scaling 需要多 GPU，并让每个 MPI rank 绑定独立 GPU。



④ np=8 时 Pinned Memory 将总时间从 28.16s 降至 9.10s，稳定性与性能均显著改善，消除了 pageable 内存导致的异常延迟。

同步CUDA的迭代顺序：SOR(red/black) → Ghost D2H → MPI_Sendrecv → Ghost H2D → next iteration，通信阶段会阻塞下一轮计算，存在重叠空间。

异步 Stream：计算/通信/IO 三流水线

三条独立 CUDA Stream 并行执行

compute_stream

SOR Kernel 内部点计算

comm_stream

D2H → MPI → H2D 边界
交换

io_stream

IO 写盘（仅末步执行）

性能对比

网格	同步 CUDA	异步 CUDA
512×512 (1000步)	0.8745 s	0.8169 s
1024×1024 (1900步)	1.6386 s	1.5916 s

异步版获得约 5-6% 提升，优势有限但验证了 Stream 并发模型的正确性。

CUDA Event 精确同步依赖关系，隐藏通信与 IO 延迟。

$np \geq 4$ 二维分解下，residual 出现爆炸或假零！

多子域 CUDA 稳定性优化

可能原因

- 跨子域 Red-Black SOR 存在 halo 信息延迟，导致边界数据不一致

优化前后对比

场景	修复前	修复后
1024×1024 np=4	50000步， 残差爆炸	1900步， 正常收敛
1024×1024 np=8	residual 假零	1900步， 正常收敛

优化方案

- residual Kernel 检测到 NaN/Inf 时标记为 INFINITY
- 每个 color 更新后立即执行 halo exchange，保证同步性

⚠ 代价：halo exchange 次数增加，np=4/8 总时间仍较高，需进一步优化通信重叠。

⚠ 硬件条件限制：单机多进程共享 GPU 时 Strong Scaling 效率有限，多节点多 GPU 是下一步优化方向。

前五轮保留MPI+CUDA架构，适合展示MPI域分解、halo exchange和多GPU扩展思路。
但在当前硬件上只有一块GPU，多个MPI rank共享同一GPU会引入额外context和通信开销。

单GPU全局CUDA Kernel

- 不调用MPI_Init，不做MPI域分解，不做ghost exchange
- 全局 $(nx+2) \times (ny+2)$ 网格直接放在一块GPU上
- Red-Black SOR使用两个kernel更新红点和黑点
- residual仍在GPU上归约

代码实现：

```
while (resid > tol && iter < max_iters) {
    sor_single_kernel<<<grid, block>>>(d_u, nx, ny, stride, 0, omega);
    sor_single_kernel<<<grid, block>>>(d_u, nx, ny, stride, 1, omega);
    iter++;

    if (iter % 50 == 0) {
        residual_single_kernel<<<grid, block>>>(d_u, d_residual, nx, ny, stride);
        cudaMemcpy(&resid, d_residual, sizeof(double), cudaMemcpyDeviceToHost);
    }
}
```

网格	MPI+CUDA np=1	单GPU CUDA	改善
256×256	0.0305 s	0.0172 s	43.6%
512×512	0.0892 s	0.0701 s	21.4%
1024×1024	0.4769 s	0.4360 s	8.6%
2048×2048	3.2303 s	3.2459 s	约同

结论：

- 小中规模下，去掉MPI框架和halo exchange后，单GPU全局版更快
- 2048×2048时两者接近，说明大规模下kernel计算占主导，MPI np=1的额外开销被摊薄
- 该版本不是替代MPI+CUDA，而是作为单GPU硬件条件下的最佳性能参考

Strong & Weak Scaling

三类评估方案

CPU Scaling

CPU , np=1/2/4/8 , 验证 MPI 域分解效率

Single-GPU MPI Sharing

CUDA , np=1/2/4/8 , 分析单 GPU 被多进程共享时的竞争开销

Single-GPU Size Scaling

CUDA , np=1 , 改网格规模 , 验证 GPU Kernel 随规模增长的收益

CPU Strong Scaling : 固定1024×1024

np	CPU SOR	Speedup
1	12.7470 s	1.00×
2	7.7941 s	1.64×
4	4.4429 s	2.87×
8	2.5610 s	4.98×

CPU Weak Scaling : 每进程约256×256

np	总网格	CPU SOR
1	256×256	0.1771 s
2	362×362	0.3234 s
4	512×512	0.5388 s
8	724×724	0.8651 s

■ CPU (s) ■ CUDA (s)

加速比 (np=1)

加速比随网格规模递增：

6.34 × → 14.31 × → 26.23 ×

GPU 优势在较大规模问题上体现！



总结与收获

✓ 算法层面

SOR + 最优 ω 是解 2D Laplace 的最佳迭代选择，收敛速度比 Jacobi 更快

✓ 并行架构

MPI 两方向 2D 笛卡尔分解 + Halo Exchange，MPI 并行 IO 正确性得到验证

✓ GPU 加速

CUDA Red-Black SOR Kernel + Pinned Memory，相对 CPU 最高 26.9× 加速

✓ 异步优化

三个 CUDA Stream 重叠计算/通信/IO，更大规模下收益更显著

✓ 惨痛教训：**git -rm**如果误操作会直接删除工作区和暂存区的文件，且未经过回收站，可能导致未提交或未备份的代码永久丢失。

谢谢

Thank you for your attention!